

Efficient On-device Training via Gradient Filtering

Yuedong Yang Guihong Li Radu Marculescu
 The University of Texas at Austin

{albertyoung, lgh, radum}@utexas.edu

Abstract

*Despite its importance for federated learning, continuous learning and many other applications, on-device training remains an open problem for EdgeAI. The problem stems from the large number of operations (e.g., floating point multiplications and additions) and memory consumption required during training by the back-propagation algorithm. Consequently, in this paper, we propose a new gradient filtering approach which enables on-device CNN model training. More precisely, our approach creates a special structure with fewer unique elements in the gradient map, thus significantly reducing the computational complexity and memory consumption of back propagation during training. Extensive experiments on image classification and semantic segmentation with multiple CNN models (e.g., MobileNet, DeepLabV3, UPerNet) and devices (e.g., Raspberry Pi and Jetson Nano) demonstrate the effectiveness and wide applicability of our approach. For example, compared to SOTA, we achieve up to $19\times$ speedup and 77.1% memory savings on ImageNet classification with only 0.1% accuracy loss. Finally, our method is easy to implement and deploy; over $20\times$ speedup and 90% energy savings have been observed compared to highly optimized baselines in MKLDNN and CUDNN on NVIDIA Jetson Nano. Consequently, our approach opens up a new direction of research with a huge potential for on-device training.*¹

1. Introduction

Existing approaches for on-device training are neither efficient nor practical enough to satisfy the resource constraints of edge devices (Figure 1). This is because these methods do not properly address a fundamental problem in on-device training, namely *the computational and memory complexity of the back-propagation (BP) algorithm*. More precisely, although the architecture modification [6] and layer freezing [18, 20] can help skipping the BP for some layers, for other layers, the complexity remains high. Gra-

dient quantization [4, 7] can reduce the cost of arithmetic operations but cannot reduce the number of operations (e.g., multiplications); thus, the speedup in training remains limited. Moreover, gradient quantization is not supported by existing deep-learning frameworks (e.g., CUDNN [9], MKLDNN [1], PyTorch [25] and Tensorflow [2]). To enable on-device training, there are two important questions must be addressed:

- *How can we reduce the computational complexity of back propagation through the convolution layers?*
- *How can we reduce the data required by the gradient computation during back propagation?*

In this paper, we propose *gradient filtering*, a new research direction, to address both questions. By addressing the first question, we reduce the computational complexity of training; by addressing the second question, we reduce the memory consumption.

In general, the gradient propagation through a convolution layer involves multiplying the gradient of the output variable with a Jacobian matrix constructed with data from either the input feature map or the convolution kernel. We aim at simplifying this process with the new gradient filtering approach proposed in Section 3. Intuitively, if the gradient map w.r.t. the output has the same value for all entries, then the computation-intensive matrix multiplication can be greatly simplified, and the data required to construct the Jacobian matrix can be significantly reduced. Thus, our gradient filtering can approximate the gradient w.r.t. the output by creating a new gradient map with a special (*i.e.*, spatial) structure and fewer unique elements. By doing so, the gradient propagation through the convolution layers reduces to cheaper operations, while the data required (hence memory) for the forward propagation also lessens. Through this filtering process, we trade off the gradient precision against the computation complexity during BP. We note that gradient filtering does not necessarily lead to a worse precision, *i.e.*, models sometimes perform better with filtered gradients when compared against models trained with vanilla BP.

In summary, our contributions are as follows:

¹Code: <https://github.com/SLDGroup/GradientFilter-CVPR23>

- We propose *gradient filtering*, which reduces the computation and memory required for BP by more than two orders of magnitude compared to the exact gradient calculation.
- We provide a rigorous error analysis which shows that the errors introduced by the gradient filtering have only a limited influence on model accuracy.
- Our experiments with multiple CNN models and computer vision tasks show that we can train a neural network with significantly less computation and memory costs, with only a marginal accuracy loss compared to baseline methods. Side-by-side comparisons against other training acceleration techniques also suggest the effectiveness of our method.
- Our method is easy to deploy with highly optimized deep learning frameworks (*e.g.*, MKLDNN [1] and CUDNN [9]). Evaluations on resource-constrained edge (Raspberry Pi and Jetson Nano) and high-performance devices (CPU/GPU) show that our method is highly suitable for real life deployment.

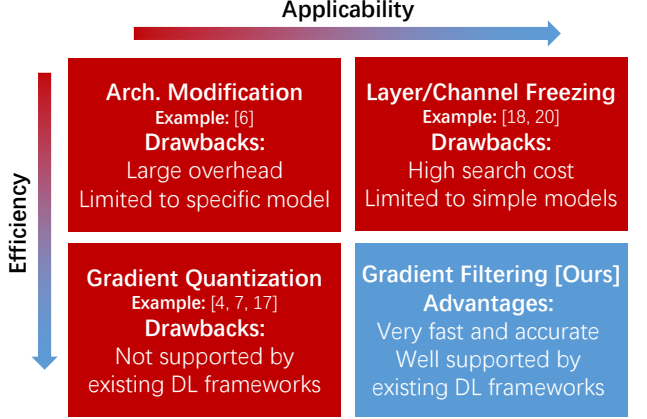
The paper is organized as follows. Section 2 reviews relevant work. Section 3 presents our method in detail. Section 4 discusses error analysis, computation and memory consumption. Experimental results are presented in Section 5. Finally, Section 6 summarizes our main contributions.

2. Related Work

Architecture Modification: Authors of [6] propose to attach small branches to the original neural network. During training, the attached branches and biases in the original model are updated. Though memory consumption is reduced, updating these branches still needs gradient propagation through the entire network; moreover, a large computational overhead for inference is introduced.

Layer Freezing: Authors of [18, 20] propose to only train parts of the model. [18] makes layer selection based on layer importance metrics, while [20] uses evolutionary search. However, the layers selected by all these methods are typically computationally heavy layers (*e.g.*, the last few layers in ResNet [14]) which consume most of the resources. Thus, the speedup achieved by these approaches is limited.

Gradient Quantization: [3, 5] quantize gradient after back-propagation, which means these methods cannot accelerate the training on a single device. Work in [4, 7, 15, 17, 28, 29, 33] accelerates training by reducing the cost for every arithmetic operation. However, these methods do not reduce the number of operations, which is typically huge for SOTA CNNs, so their achievable speedup is limited. Also, all these methods are not supported by the popular deep learning frameworks [1, 2, 9, 25].



Orthogonal Research Directions for On-device Training

Figure 1. Matrix of orthogonal directions for on-device training. “Arch” is short for “architecture”. Our approach opens up a new direction of research for on-device training for EdgeAI.

In contrast to the prior work, our method opens up a new research direction. More precisely, we reduce the number of computations and memory consumption required for training a single layer via gradient filtering. Thus, our method can be combined with any of the methods mentioned above. For example, in Section H in the Supplementary, we illustrate how our method can work together with the gradient quantization methods to enable a higher speedup.

3. Proposed Method

In this section, we introduce our gradient filtering approach to accelerate BP. To this end, we target the most computation and memory heavy operation, *i.e.*, convolution (Figure 2(a)). Table 1 lists some symbols we use.

C_x	Number of channels of x
W_x, H_x	Width and height of x
θ	Convolution kernel
θ'	Rotated θ , <i>i.e.</i> , $\theta' = \text{rot180}(\theta)$
r	Patch size ($r \times r$)
g_x, g_y, g_θ	Gradients w.r.t. x, y, θ
$\tilde{g}_y$	Approximated gradient g_y
$\tilde{x}, \tilde{\theta}'$	Sum of x and θ' over spatial dimensions (height and width)
$x[n, c_i, h, w]$	Element for feature map x at batch n , channel c_i , pixel (h, w)
$\theta[c_o, c_i, u, v]$	Element for convolution kernel θ at output channel c_o , input channel c_i , position (u, v)

Table 1. Table of symbols we use.

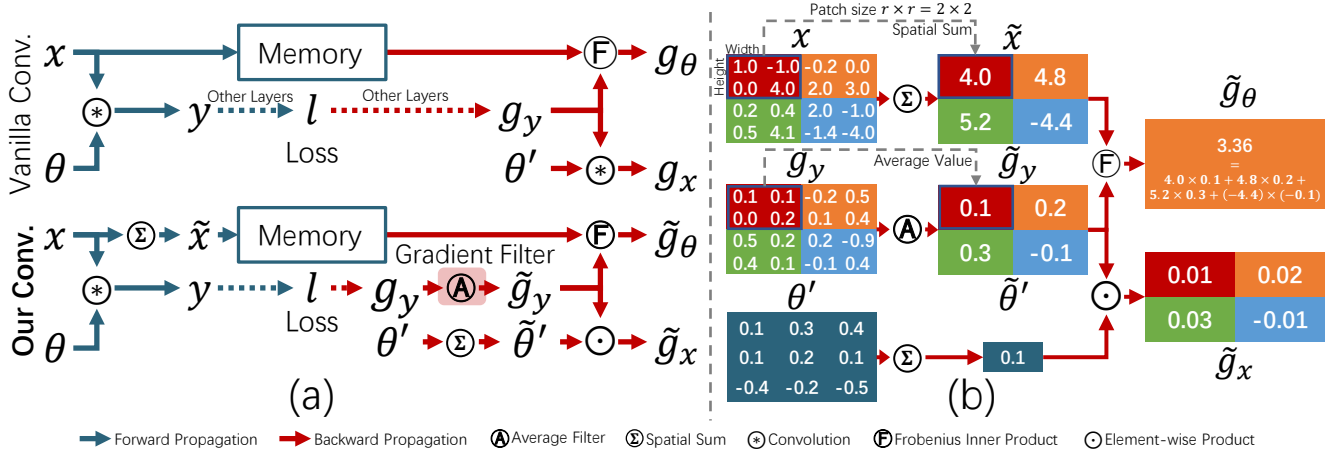


Figure 2. (a) Computation procedures for vanilla training method (upper) and our method (lower). (b) Example of gradient propagation with gradient filtering. Numbers in this example are chosen randomly for illustration purposes. In this case, the patch size selected for the gradient filter is 2×2 . Thus, the 4×4 gradient map g_y is approximated by $\tilde{g}_y$, which has four 2×2 patches with one unique value for each patch. Also, input feature map x and mirrored convolution kernel θ' are spatially summed to $\tilde{x}$ and $\tilde{\theta}'$. Since $\tilde{x}$ has fewer unique values than x , memory consumption is reduced. Finally, with $\tilde{g}_y$, $\tilde{x}$ and $\tilde{\theta}$, we compute the gradient w.r.t. kernel and input feature map with much fewer operations than the standard back propagation method.

3.1. Problem Setup

The computations for both forward and backward paths are shown in Figure 2(a). For the standard (vanilla) approach (upper Figure 2(a)), starting with input x , the forward propagation convolves the input feature map x with kernel θ and returns output y , which is further processed by the other layers in the neural network (dotted arrow) until the loss value l is calculated. As shown in Figure 2(a), the BP of the convolution layer starts with the gradient map w.r.t. output y (g_y). The gradient w.r.t. input (g_x) is calculated by convolving g_y with the *rotated* convolution kernel θ' , i.e., $g_x = g_y \otimes \text{rot}180(\theta) = g_y \otimes \theta'$. The gradient w.r.t. convolution kernel, namely g_θ , is calculated with the Frobenius inner product [16] between x and g_y , i.e., $g_\theta = g_y \otimes x$.

The lower half of Figure 2(a) shows our method, where several changes are made: We introduce the gradient filter “(A)” after g_y to generate the approximate gradient for BP. Also, instead of using the accurate x and θ' values for gradient computation, we sum over spatial dimensions (height and width dimensions), i.e., $\tilde{x}$ and $\tilde{\theta}'$, respectively. Finally, the convolution layer now multiplies the approximate gradient $\tilde{g}_y$ with spatial kernel $\tilde{\theta}'$ instead of convolving with it to calculate $\tilde{g}_x$. Figure 2(b) shows an example of gradient propagation with our gradient filter.

3.2. Preliminary Analysis

Consider the vanilla BP for convolution in Figure 2(a). Equation (1) shows the number of computations (#FLOPs) required to calculate g_x given g_y :

$$\text{\#FLOPs} = 2C_x C_y \cdot W_y H_y \cdot W_\theta H_\theta \quad (1)$$

The computation requirements in Equation (1) belong to three categories: number of channels, number of *unique elements* per channel in the gradient map, and *kernel size*. Our method focuses on the last two categories.

i. Unique elements: ($W_y H_y$) represents the number of unique elements per channel in the gradient w.r.t. output variable y (g_y). Given the high-resolution images we use, this term is huge, so if we manage to reduce the number of unique elements in the spatial dimensions (height and width), the computations required are greatly reduced too.

ii. Kernel size: ($W_\theta H_\theta$) represents the number of unique elements in the convolution kernel. If the gradient g_y has some special structure, for example $g_y = 1_{H_y \times W_y} \cdot v$ (i.e., every element in g_y has the same value v), then the convolution can be simplified to $(\sum \theta') v 1_{H_y \times W_y}$ (with boundary elements ignored). With such a special structure, only one multiplication and $(W_\theta H_\theta - 1)$ additions are required. Moreover, $\sum \theta'$ is independent of data so the result can be shared across multiple images until θ gets updated.

3.3. Gradient Filtering

To reduce the number of unique elements and create the special structure in the gradient map, we apply the gradient filter after the gradient w.r.t. output (g_y) is provided. During the backward propagation, the gradient filter “(A)” approximates the gradient g_y by spatially cutting the gradient map into $r \times r$ -pixel patches and then replacing all elements in each patch with their average value (Figure 2(b)):

$$\tilde{g}_y[n, c_o, h, w] = \frac{1}{r^2} \sum_{i=\lfloor h/r \rfloor r}^{\lceil h/r \rceil r} \sum_{j=\lfloor w/r \rfloor r}^{\lceil w/r \rceil r} g_y[n, c_o, i, j] \quad (2)$$

For instance in Figure 2(b), we replace the 16 distinct values in the gradient map g_y with 4 average values in $\tilde{g}_y$. So given a gradient map g_y with N images per batch, C channels, and $H \times W$ pixels per channel, the gradient filter returns a structured approximation of the gradient map containing only $N \times C \times \lceil \frac{H}{r} \rceil \times \lceil \frac{W}{r} \rceil$ blocks, with *one unique value per patch*. We use this matrix of unique values to represent the approximate gradient map $\tilde{g}_y$, as shown in Figure 2(b).

3.4. Back Propagation with Gradient Filtering

We describe now the computation procedure used after applying the gradient filter. Detailed derivations are provided in Supplementary Section B.

Gradient w.r.t. input: The gradient w.r.t. input is calculated by convolving θ' with g_y (Figure 2(a)). With the approximate gradient $\tilde{g}_y$, this convolution simplifies to:

$$\tilde{g}_x[n, c_i, h, w] = \sum_{c_o} \tilde{g}_y[n, c_o, h, w] \odot \tilde{\theta}'[c_o, c_i] \quad (3)$$

where $\tilde{\theta}'[c_o, c_i] = \sum_{u,v} \theta'[c_o, c_i, u, v]$ is the spatial sum of convolution kernel θ , as shown in Figure 2(b).

Gradient w.r.t. kernel: The gradient w.r.t. the kernel is calculated by taking the Frobenius inner product between x and g_y , i.e., $g_\theta[c_o, c_i, u, v] = x \oplus g_y$, namely:

$$g_\theta[c_o, c_i, u, v] = \sum_{n,i,j} x[n, c_i, i+u, j+v] g_y[n, c_o, i, j] \quad (4)$$

With the approximate gradient $\tilde{g}_y$, the operation can be simplified to:

$$\tilde{g}_\theta[c_o, c_i, u, v] = \sum_{n,i,j} \tilde{x}[n, c_i, i, j] \tilde{g}_y[n, c_o, i, j] \quad (5)$$

with $\tilde{x}[n, c_i, i, j] = \sum_{h=\lfloor i/r \rfloor r}^{\lceil i/r \rceil r} \sum_{w=\lfloor j/r \rfloor r}^{\lceil j/r \rceil r} x[n, c_i, h, w]$. As shown in Figure 2(b), $\tilde{x}[n, c_i, i, j]$ is the spatial sum of x elements in the same patch containing pixel (i, j) .

4. Analyses of Proposed Approach

In this section, we analyze our method from three perspectives: gradient filtering approximation error, computation reduction, and memory cost reduction.

4.1. Error Analysis of Gradient Filtering

We prove that the approximation error introduced by our gradient filtering is bounded during the gradient propagation. Without losing generality, we consider that all variables have only one channel, i.e., $C_{x_0} = C_{x_1} = 1$.

Proposition 1: For any input-output channel pair (c_o, c_i) in the convolution kernel θ , assuming the DC component has the largest energy value compared to all components in

the spectrum², then the signal-to-noise-ratio (SNR) of $\tilde{g}_x$ is greater than SNR of $\tilde{g}_y$.

Proof: We use G_x, G_y and Θ to denote the gradients g_x, g_y and the convolution kernel θ in the frequency domain; $G_x[u, v]$ is the spectrum value at frequency (u, v) and δ is the 2D discrete Dirichlet function. To simplify the discussion, we consider only one patch of size $r \times r$.

The gradient returned by the gradient filtering can be written as:

$$\tilde{g}_y = \frac{1}{r^2} 1_{r \times r} \otimes g_y \quad (6)$$

where $\otimes$ denotes convolution. By applying the discrete Fourier transformation, Equation (6) can be rewritten in the frequency domain as:

$$\tilde{G}_y[u, v] = \frac{1}{r^2} \delta[u, v] G_y[u, v] \quad (7)$$

$\tilde{g}_y$ is the approximation of g_y (i.e., the ground truth for $\tilde{g}_y$ is g_y), and the SNR of $\tilde{g}_y$ equals to:

$$\begin{aligned} \text{SNR}_{\tilde{g}_y} &= \frac{\sum_{(u,v)} G_y^2[u, v]}{\sum_{(u,v)} (G_y[u, v] - \frac{1}{r^2} \delta[u, v] G_y[u, v])^2} \\ &= (1 - \frac{2r^2 - 1}{r^4} \frac{G_y^2[0, 0]}{\sum_{(u,v)} G_y^2[u, v]})^{-1} \end{aligned} \quad (8)$$

For the convolution layer, the gradient w.r.t. the approximate variable $\tilde{x}$ in the frequency domain is³:

$$\begin{aligned} \tilde{G}_x[u, v] &= \Theta[-u, -v] \tilde{G}_y[u, v] \\ &= \frac{1}{r^2} \Theta[-u, -v] \delta[u, v] G_y[u, v] \end{aligned} \quad (9)$$

and its ground truth is:

$$G_x[u, v] = \Theta[-u, -v] G_y[u, v] \quad (10)$$

Similar to Equation (8), the SNR of $\tilde{g}_x$ is:

$$\text{SNR}_{\tilde{g}_x} = (1 - \frac{2r^2 - 1}{r^4} \frac{(\Theta[0, 0] G_y[0, 0])^2}{\sum_{(u,v)} (\Theta[u, v] G_y[u, v])^2})^{-1} \quad (11)$$

Equation (11) can be rewritten as:

$$\begin{aligned} \frac{r^4(1 - \text{SNR}_{\tilde{g}_x}^{-1})}{2r^2 - 1} &= \frac{(\Theta[0, 0] G_y[0, 0])^2}{\sum_{(u,v)} (\Theta[-u, -v] G_y[u, v])^2} \\ &= \frac{G_y^2[0, 0]}{\sum_{(u,v)} (\frac{\Theta[-u, -v]}{\Theta[0, 0]} G_y[u, v])^2} \end{aligned} \quad (12)$$

Furthermore, the main assumption (i.e., the DC component dominates the frequency spectrum of Θ) can be written as:

$$\Theta^2[0, 0] / \max_{(u,v) \neq (0,0)} \Theta^2[u, v] \geq 1 \quad (13)$$

²As a reminder, the energy of a signal is the sum of energy of the DC component and the energy of its AC components.

³Because g_y is convolved with the **rotated** kernel θ' , in the frequency domain, we use $\Theta[-u, -v]$ instead of $\Theta[u, v]$.

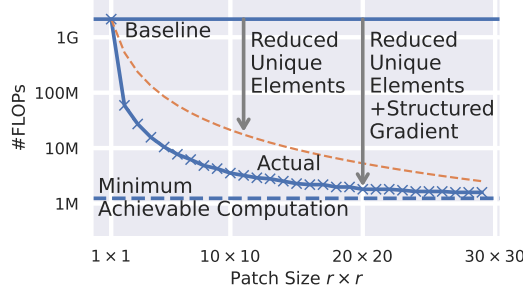


Figure 3. Computation analysis for a specific convolution layer⁴. Minimum achievable computation is given in Equation (16). By reducing the number of unique elements, computations required by our approach drop to about $1/r^2$ compared with the standard BP method. By combining it with structured gradient map, computations required by our approach drop further, getting very close to the theoretical limit.

that is, $\forall(u, v), \frac{\Theta^2[-u, -v]}{\Theta^2[0, 0]} \leq 1$; thus, by combining Equation (12) and Equation (13), we have:

$$\begin{aligned} \frac{G_y^2[0, 0]}{\sum_{(u, v)} \left(\frac{\Theta[-u, -v]}{\Theta[0, 0]} G_y[u, v] \right)^2} &\geq \frac{G_y^2[0, 0]}{\sum_{(u, v)} (G_y[u, v])^2} \\ \Leftrightarrow \frac{r^4(1 - \text{SNR}_{\tilde{g}_y}^{-1})}{2r^2 - 1} &\geq \frac{r^4(1 - \text{SNR}_{\tilde{g}_y}^{-1})}{2r^2 - 1} \end{aligned} \quad (14)$$

which means that: $\text{SNR}_{\tilde{g}_x} \geq \text{SNR}_{\tilde{g}_y}$. This completes our proof for error analysis. ■

In conclusion, as the gradient propagates through the network, the noise introduced by our gradient filter becomes weaker compared to the real gradient signal. This property ensures that the error in gradient has only a limited influence on the quality of BP. We validate Proposition 1 later in the experimental section.

4.2. Computation and Overhead Analysis

In this section, we analyse the computation required to compute g_x , the gradient w.r.t. input x . Figure 3 compares the computation required to propagate the gradient through this convolution layer under different patch sizes $r \times r$. A patch size 1×1 means the vanilla BP algorithm which we use as the baseline. As discussed in the preliminary analysis section (Section 3.2), two terms contribute to the computation savings: fewer unique elements in the gradient map and the structured gradient map.

Fewer unique elements: In vanilla BP, there are $H_y W_y$ unique elements in the gradient map. After applying gradient filtering with a patch size $r \times r$, the number of unique

elements reduces to only $\lceil \frac{H_y}{r} \rceil \lceil \frac{W_y}{r} \rceil$. This reduction contributes the most to the savings in computation (orange line in Figure 3).

Structured Gradient Map: By creating the structured gradient map, the convolution over the gradient map $\tilde{g}_y$ is simplified to the element-wise multiplication and channel-wise addition. Computation is thus reduced to $(H_\theta W_\theta)^{-1}$ of its original value. For instance, the example convolution layer in Figure 3 uses a 3×3 convolution kernel so around 89% computations are removed. The blue line in Figure 3 shows the #FLOPs after combining both methods. Greater reduction is expected when applying our method with larger convolution kernels. For instance, FastDepth [30] uses 5×5 convolution kernel so as much as 96% reduction in computation can be achieved, in principle.

Minimum Achievable Computation: With the two reductions mentioned above, the computation required to propagate the gradient through the convolution layer is:

$$\#FLOPs(r) = \lceil \frac{H_y}{r} \rceil \lceil \frac{W_y}{r} \rceil C_x (2C_y - 1) + o(H_y W_y) \quad (15)$$

where $o(H_y W_y)$ is a constant term which is independent of r and negligible compared to $H_y W_y$. When the patch is as large as the feature map, our method reaches the minimum achievable computation (blue dashed line in Figure 3):

$$\min_r \#FLOPs(r) = 2C_x C_y - C_x + o(H_y W_y) \quad (16)$$

In this case, each channel of the gradient map is represented with a single value, so the computation is controlled by the number of input and output channels.

Overhead: The overhead of our approach comes from approximating the feature map x , gradient g_y , and kernel θ . As the lower part of Figure 2(a) shows, the approximation for x is considered as part of forward propagation, while the other two as back propagation. Indeed, with the patch size r , the ratio of forward propagation overhead is about $1/(2C_o W_\theta H_\theta)$, while the ratio of backward propagation overhead is about $(r^2 - 1)/(2C_x)$.

Given the large number of channels and spatial dimensions in typical neural networks, both overhead values take less than 1% computation in the U-Net example above.

4.3. Memory Analysis

As Figure 2(a) shows, the standard back propagation for a convolution layer relies on the input feature map x , which needs to be stored in memory during forward propagation. Since every convolution layer requiring gradient for its kernel needs to save a copy of feature map x , the memory consumption for storing x is huge. With our method, we simplify the feature map x to approximated $\tilde{x}$, which has only $\lceil \frac{H_x}{r} \rceil \lceil \frac{W_x}{r} \rceil$ unique elements for every channel. Thus, by saving only these unique values, our method achieves around $(1 - \frac{1}{r^2})$ memory savings, overall.

⁴The layer is from U-Net [26]. The size of the input is assumed to be 120×160 pixels with 192 channels; the output has the same resolution, but with only 64 channels. The kernel size of the convolution layer is 3×3 . Analysis for ResNet is included in the supplementary material.

MobileNetV2 [27]	#Layers	Accuracy	FLOPs	Mem	ResNet-18 [14]	#Layers	Accuracy	FLOPs	Mem
No Finetuning	0	4.2	0	0	No Finetuning	0	4.7	0	0
Vanilla	All	75.1	1.13G	24.33MB	Vanilla	All	73.1	5.42G	8.33MB
BP	2	63.1	113.68M	245.00KB	BP	2	70.4	489.20M	196.00KB
	4	62.2	160.00M	459.38KB		4	72.3	1.14G	490.00KB
TinyTL [6]	N/A	60.2	663.51M	683.00KB	TinyTL [6]	N/A	69.2	3.88G	1.76MB
Ours	2	63.1	39.27M	80.00KB	Ours	2	68.6	28.32M	64.00KB
	4	63.4	53.96M	150.00KB		4	68.5	61.53M	112.00KB
MCUNet [19]	#Layers	Accuracy	FLOPs	Mem	ResNet-34 [14]	#Layers	Accuracy	FLOPs	Mem
No Finetune	0	4.1	0	0	No Finetune	0		0	0
Vanilla	All	68.5	231.67M	9.17MB	Vanilla	All	70.8	11.17G	13.11MB
BP	2	62.1	18.80M	220.50KB	BP	2	69.6	489.20M	196.00KB
	4	64.9	33.71M	312.38KB		4	72.3	1.21G	392.00KB
TinyTL [6]	N/A	53.1	148.01M	571.5KB	TinyTL [6]	N/A	72.9	8.03G	2.95MB
Ours	2	61.8	6.34M	72.00KB	Ours	2	68.6	28.32M	64.00KB
	4	64.4	11.01M	102.00KB		4	70.6	64.07M	128.00KB

Table 2. Experimental results for ImageNet classification with four neural networks (MobileNet-V2, ResNet18/34, MCUNet). “#Layers” is short for “the number of *active* convolutional layers”. For example, #Layers equals to 2 means that only the last two convolutional layers are trained. For memory consumption, we only consider the memory for input feature x . Strategy “No Finetuning” shows the accuracy on new datasets without finetuning the pretrained model. Since TinyTL [6] changes the architecture, “#Layers” is not applicable (N/A).

PSPNet [32]	#Layers	GFLOPs	mIoU	mAcc	PSPNet-M [32]	#Layers	GFLOPs	mIoU	mAcc	FCN [21]	#Layers	GFLOPs	mIoU	mAcc
Calibration	0	0	12.86	19.74	Calibration	0	0	14.20	20.46	Calibration	0	0	10.95	15.69
Vanilla	All	166.5	55.01	68.02	Vanilla	All	42.4	48.48	61.48	Vanilla	All	170.3	45.22	58.80
BP	5	15.0	39.54	51.86	BP	5	12.22	36.35	47.09	BP	5	59.5	27.41	37.90
	10	110.6	53.15	67.10		10	22.46	46.01	58.70		10	100.9	43.87	57.58
Ours	5	0.14	39.34	51.86	Ours	5	0.11	36.14	46.86	Ours	5	0.58	27.42	37.88
	10	0.79	50.88	64.73		10	0.76	44.90	57.50		10	0.96	36.30	48.82
DLV3 [8]	#Layers	GFLOPs	mIoU	mAcc	DLV3-M [8]	#Layers	GFLOPs	mIoU	mAcc	UPerNet [31]	#Layers	GFLOPs	mIoU	mAcc
Calibration	0	0	13.95	20.62	Calibration	0	0	21.96	36.15	Calibration	0	0	14.71	21.82
Vanilla	All	151.2	58.32	71.72	Vanilla	All	54.4	55.66	68.95	Vanilla	All	541.0	64.88	77.13
BP	5	18.0	40.85	53.16	BP	5	14.8	38.21	49.35	BP	5	503.9	47.93	61.67
	10	102.0	54.65	68.64		10	33.1	47.95	61.49		10	507.6	48.83	63.02
Ours	5	0.31	33.09	44.33	Ours	5	0.26	35.47	46.35	Ours	5	1.97	47.04	60.44
	10	2.96	47.11	60.28		10	1.40	45.53	58.99		10	2.22	48.00	62.07

Table 3. Experimental results for semantic segmentation task on augmented Pascal VOC12 dataset [8]. Model name with postfix “M” means the model uses MobileNetV2 as backbone, otherwise ResNet18 is used. “#Layers” is short for “the number of *active* convolutional layers” that are trained. All models are pretrained on Cityscapes dataset [11]. Strategy “Calibration” shows the accuracy when only the classifier and normalization statistics are updated to adapt different numbers of classes between augmented Pascal VOC12 and Cityscapes.

5. Experiments

Our experimental section consists of theoretical and practical evaluations. Sections 5.2-5.4 show the theoretical advantages of our method on image classification and semantic segmentation tasks with implementation-agnostic metrics (e.g., accuracy, FLOPs). Then, in Section 5.5, we show how these theoretical advantages translate into practical advantages (i.e., speedup and memory savings) on real edge devices.

5.1. Experimental Setup

Classification: Following [24], we split every dataset into two highly non-i.i.d. partitions with the same size. Then, we pretrain our models on the first partition with a vanilla training strategy, and finetune the model on the other partition with different configurations for the training strat-

egy (i.e., with/without gradient filtering, hyper-parameters, number of convolution layers to be trained). More details (e.g., hyper-parameters) are in the Supplementary.

Segmentation: Models are pretrained on Cityscapes [11] by MMSegmentation [10]. Then, we calibrate and finetune these models with different training strategies on the augmented Pascal-VOC12 dataset following [8], which is the combination of Pascal-VOC12 [12] and SBD [13]. More details are included in the supplementary material.

On-device Performance Evaluation: For CPU performance evaluation, we implement our method with MKLDNN [1] (a.k.a. OneDNN) v2.6.0 and compare it with the convolution BP method provided by MKLDNN. We test on three CPUs, namely Intel 11900KF, Quad-core Cortex-A72 (Jetson Nano) and Quad-core Cortex-A53 (Raspberry Pi-3b). For GPU performance evaluation, we implement our method on CUDNN v8.2 [9] and compare with the BP

method provided by CUDNN. We test on two GPUs, RTX 3090Ti and the edge GPU on Jetson Nano. Since both MKLDNN and CUDNN only support float32 BP, we test float32 BP only. Additionally, for the experiments on Jetson Nano, we record the energy consumption for CPU and GPU with the embedded power meter. More details (*e.g.*, frequency) are included in the supplementary material.

5.2. ImageNet Classification

Table 2 shows our evaluation results on the ImageNet classification task. As shown, our method significantly reduces the FLOPs and memory required for BP, with very little accuracy loss. For example, for ResNet34, our method achieves $18.9\times$ speedup with 1.7% accuracy loss when training four layers; for MobileNetV2, we get a 1.2% better accuracy with $3.0\times$ speedup and $3.1\times$ memory savings. These results illustrate the effectiveness of our method. On most networks, TinyTL has a lower accuracy while consuming more resources compared to the baselines methods.

5.3. Semantic Segmentation

Table 3 shows our evaluation results on the augmented Pascal-VOC12 dataset. On a wide range of networks, our method constantly achieves significant speedup with marginal accuracy loss. For the large network UPerNet, our method achieves $229\times$ speedup with only 1% mIoU loss. For the small network PSPNet, our method speeds up training by $140\times$ with only 2.27% mIoU loss. This shows the effectiveness of our method on a dense prediction task.

5.4. Hyper-Parameter Selection

Figure 4 shows our experimental results for ResNets under different hyper-parameter selection, *i.e.* number of convolution layers and patch size of gradient filter $r \times r$. Of note, the y-axis (MFLOPs) in Figure 4 is log scale. More results are included in Supplementary Section G. We highlight three qualitative findings in Figure 4:

- a. For a similar accuracy, our method greatly reduces the number of operations (1 to 2 orders of magnitude), while for a similar number of computations, our method achieves a higher accuracy (2% to 5% better).

This finding proves the effectiveness of our method.

- b. Given the number of convolution layers to be trained, the more accurate method returns a better accuracy. Baseline (*i.e.*, standard BP) uses the most accurate gradient, Ours-R4 (BP with gradient filter with patch size 4×4) uses the least accurate gradient; thus, Baseline $>$ Ours-R2 $>$ Ours-R4.

This finding is intuitive since the more accurate method should introduce smaller noise to the BP, *e.g.*, the gradient filtering with patch size 2×2 (Ours-R2) introduces less

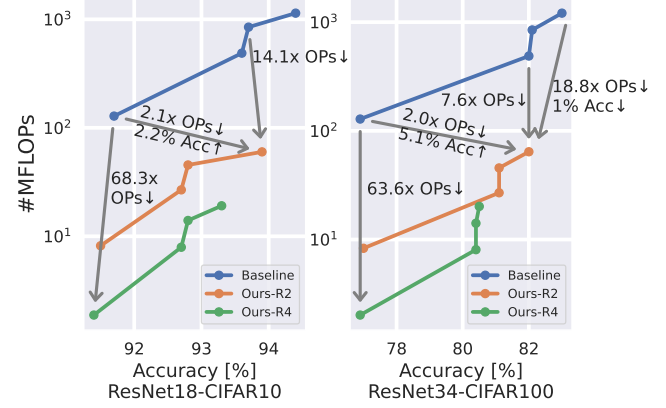


Figure 4. Computation (#MFLOPs, log scale) and model accuracy [%] under different hyper-parameter selection. “Baseline” means vanilla BP; “Ours-R2/4” uses gradient filtering with patch size $2 \times 2/4 \times 4$ during BP.

noise than with patch size 4×4 (Ours-R4). In Figure 5, we evaluate the relationship between accuracy and noise level introduced by gradient filtering. With a higher SNR (*i.e.*, a lower noise level), a better accuracy is achieved.

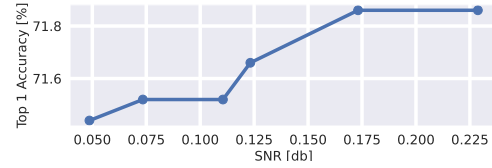


Figure 5. Relationship between accuracy and noise level introduced by the gradient filtering. As shown, accuracy increases as the SNR increases, *i.e.*, noise level decreases.

- c. Given the number of computations, the less accurate method returns the better accuracy by training more layers, *i.e.*, Ours-R4 $>$ Ours-R2 $>$ baseline.

This finding suggests that for neural network training with relatively low computational resources, training more layers with less accurate gradients is preferable than training fewer layers with more accurate gradients.

5.5. On-device Performance Evaluation

Figure 6 and Table 4 show our evaluation results on real devices. More results are included in the Supplementary Section I. As Figure 6 shows, on CPU, most convolution layers achieve speedups over $20\times$ with less than 50% memory consumption for gradient filtering with patch sizes 2×2 ; for gradient filtering with patch size 4×4 , the speedups are much higher, namely over $60\times$. On GPU, the speedup is a little bit lower, but still over $10\times$ and $25\times$, respectively. Furthermore, as Table 4 shows, our method saves over 95%

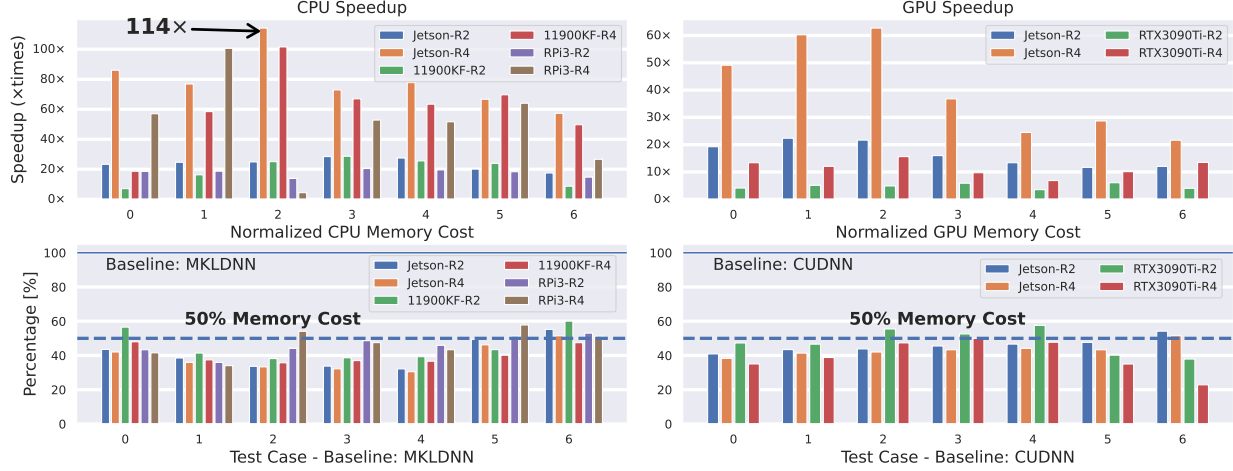


Figure 6. Speedup and normalized memory consumption results on multiple CPUs and GPUs under different test cases (*i.e.* different input sizes, numbers of channels, etc.) Detailed configuration of these test cases are included in the supplementary material. “R2”, “R4” mean using gradient filtering with 2×2 and 4×4 patch sizes, respectively. Our method achieves significant speedup with low memory consumption compared to all baseline methods. For example, on Jetson CPU with patch size 4×4 (“Jetson-R4” in left top figure), our method achieves $114\times$ speedup with only 33% memory consumption for most test cases.

Device	Patch Size	Normalized Energy Cost [STD]
Edge	2×2	4.13% [0.61%]
CPU	4×4	1.15% [0.18%]
Edge	2×2	3.80% [0.73%]
GPU	4×4	1.22% [1.10%]

Table 4. Normalized energy consumption for BP with gradient filtering for different patch sizes. Results are normalized w.r.t. the energy cost of standard BP methods. For instance, for edge CPU with a 4×4 patch, only 1.15% of energy in standard BP is used. Standard deviations are shown within brackets.

energy for both CPU and GPU scenarios, which largely resolves one of the most important constraints on edge devices. All these experiments on real devices show that our method is practical for the real deployment of both high-performance and IoT applications.

Model	Ratio	Model	Ratio
(Wide)ResNet18-152	1.462	VGG(bn)11-19	1.497
DenseNet121-201	2.278	EfficientNet b0-b7	1.240

Table 5. Evaluation of energy ratio defined in Equation (13) on models published on Torchvision. The ratio greater than 1 empirically verifies our assumption.

5.6. Main Assumption Verification

We now empirically verify the assumption that the DC component dominates the frequency spectrum of the convolution kernel (Section 4.1). To this end, we collect the en-

ergy ratio shown in Equation (13) from trained models published in Torchvision [23]. As Table 5 shows, for the convolution kernels in all these networks, we get a ratio greater than one, which means that the energy of DC components is larger than energy of all AC components. Thus, our assumption in Section 4.1 empirically holds true in practice.

6. Conclusions

In this paper, we have addressed the on-device model training for resource-constrained edge devices. To this end, a new gradient filtering method has been proposed to systematically reduce the computation and memory consumption for the back-propagation algorithm, which is the key bottleneck for efficient model training.

In Section 3, a new gradient filtering approach has been proposed to reduce the computation required for propagating gradients through the convolutional layers. The gradient filtering creates an approximate gradient feature map with fewer unique elements and a special structure; this reduces the computation by more than two orders of magnitude. Furthermore, we proved that the error introduced during back-propagation by our gradient filter is bounded so the influence of gradient approximation is limited.

Extensive experiments in Section 5 have demonstrated the efficiency and wide applicability of our method. Indeed, models can be finetuned with orders of magnitudes fewer computations, while having only a marginal accuracy loss compared to popular baseline methods.

Acknowledgements: This work was supported in part by the US National Science Foundation (NSF) grant CNS-2007284.

References

- [1] Intel® oneapi deep neural network library (onednn). <https://www.intel.com/content/www/us/en/developer/tools/oneapi/onednn.html>. 1, 2, 6
- [2] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org. 1, 2
- [3] Dan Alistarh, Demjan Grubic, Jerry Z. Li, Ryota Tomioka, and Milan Vojnovic. Qsgd: Communication-efficient sgd via gradient quantization and encoding. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17*, page 1707–1718, Red Hook, NY, USA, 2017. Curran Associates Inc. 2
- [4] Ron Banner, Itay Hubara, Elad Hoffer, and Daniel Soudry. Scalable methods for 8-bit training of neural networks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems, NIPS’18*, page 5151–5159, Red Hook, NY, USA, 2018. Curran Associates Inc. 1, 2, 11, 17, 19
- [5] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Animashree Anandkumar. signsgd: Compressed optimisation for non-convex problems. In *International Conference on Machine Learning*, pages 560–569. PMLR, 2018. 2
- [6] Han Cai, Chuang Gan, Ligeng Zhu, and Song Han. Tinytl: Reduce activations, not trainable parameters for efficient on-device learning. *arXiv preprint arXiv:2007.11622*, 2020. 1, 2, 6
- [7] Jianfei Chen, Yu Gai, Zhewei Yao, Michael W Mahoney, and Joseph E Gonzalez. A statistical framework for low-bitwidth training of deep neural networks. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 883–894. Curran Associates, Inc., 2020. 1, 2, 11, 16, 17, 19
- [8] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation. *arXiv preprint arXiv:1706.05587*, 2017. 6
- [9] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cudnn: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759*, 2014. 1, 2, 6
- [10] MMSegmentation Contributors. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. <https://github.com/open-mmlab/mms Segmentation>, 2020. 6
- [11] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 6
- [12] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88(2):303–338, June 2010. 6
- [13] Bharath Hariharan, Pablo Arbelaez, Lubomir Bourdev, Subhransu Maji, and Jitendra Malik. Semantic contours from inverse detectors. In *International Conference on Computer Vision (ICCV)*, 2011. 6
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 2, 6
- [15] Ziyang Hong and C. Patrick Yue. Efficient-grad: Efficient training deep convolutional neural networks on edge devices with gradient optimizations. *ACM Trans. Embed. Comput. Syst.*, 21(2), feb 2022. 2
- [16] Roger A Horn and Charles R Johnson. *Matrix analysis*. Cambridge university press, 2012. 3
- [17] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *J. Mach. Learn. Res.*, 18(1):6869–6898, jan 2017. 2
- [18] Yoonho Lee, Annie S Chen, Fahim Tajwar, Ananya Kumar, Huaxiu Yao, Percy Liang, and Chelsea Finn. Surgical fine-tuning improves adaptation to distribution shifts. *arXiv preprint arXiv:2210.11466*, 2022. 1, 2
- [19] Ji Lin, Wei-Ming Chen, John Cohn, Chuang Gan, and Song Han. Mcunet: Tiny deep learning on iot devices. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2020. 6
- [20] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan, and Song Han. On-device training under 256kb memory. *arXiv preprint arXiv:2206.15472*, 2022. 1, 2
- [21] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015. 6
- [22] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017. 14
- [23] Sébastien Marcel and Yann Rodriguez. Torchvision the machine-vision package of torch. In *Proceedings of the 18th ACM international conference on Multimedia*, pages 1485–1488, 2010. 8
- [24] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017. 6, 14

- [25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. 1, 2
- [26] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015. 5
- [27] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018. 6
- [28] Xiao Sun, Naigang Wang, Chia-Yu Chen, Jiamin Ni, Ankur Agrawal, Xiaodong Cui, Swagath Venkataramani, Kaoutar El Maghraoui, Vijayalakshmi (Viji) Srinivasan, and Kailash Gopalakrishnan. Ultra-low precision 4-bit training of deep neural networks. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1796–1807. Curran Associates, Inc., 2020. 2
- [29] Yue Wang, Ziyu Jiang, Xiaohan Chen, Pengfei Xu, Yang Zhao, Yingyan Lin, and Zhangyang Wang. E2-train: Training state-of-the-art cnns with over 80% energy savings. *Advances in Neural Information Processing Systems*, 32, 2019. 2
- [30] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. Fastdepth: Fast monocular depth estimation on embedded systems. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 6101–6108. IEEE, 2019. 5
- [31] Tete Xiao, Yingcheng Liu, Bolei Zhou, Yuning Jiang, and Jian Sun. Unified perceptual parsing for scene understanding. In *Proceedings of the European conference on computer vision (ECCV)*, pages 418–434, 2018. 6
- [32] Hengshuang Zhao, Jianping Shi, Xiaojuan Qi, Xiaogang Wang, and Jiaya Jia. Pyramid scene parsing network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2881–2890, 2017. 6
- [33] Kang Zhao, Sida Huang, Pan Pan, Yinghan Li, Yingya Zhang, Zhenyu Gu, and Yinghui Xu. Distribution adaptive int8 quantization for training cnns. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 3483–3491, 2021. 2